WASF204: Theory, Compliance, and Lab-Based Evaluation

Assignment Objective

The objective of this assignment is to assess students' understanding of core cloud security principles, including the Shared Responsibility Model, threat modeling (STRIDE and DREAD), data protection techniques, and compliance with cloud security standards. Through both theoretical and hands-on tasks, students will demonstrate their ability to analyze security responsibilities, apply encryption for data protection, and align practices with regulatory requirements in a cloud environment.

Description

This lab introduces students to encryption and compliance in cloud security. Students will encrypt and decrypt a file using OpenSSL to demonstrate data protection at rest, and set up a Python HTTPS server to show encryption in transit. The lab also highlights how these practices support compliance with standards like GDPR, HIPAA, and PCI-DSS.

Section A (1–2 pages)

1. Answer the following questions**:**

a) In your own words, explain what the Shared Responsibility Model means and why it is important for cloud security.

b) For each cloud model **(**IaaS, PaaS, SaaS**),** give one real world example of a security responsibility that falls on the user side and explain what could go wrong if it's neglected.

c) A company is using a SaaS email service (like Gmail or Microsoft 365). List two security responsibilities that remain with the company (not the provider), and describe how they should handle them.

Section B (1 pages)

Threat Modeling with STRIDE and DREAD

System/Component: (User Login Page of an Online Banking Web Application) base on the scenario complete the table by completing the identified threat

2. STRIDE Threats:

Threat Category	Identified Threat
Spoofing	
Tampering	
Repudiation	
Information Disclosure	
Denial of Service	
Elevation of Privilege	

3. DREAD: write the analysis the selected threat and score it

Factor	Score (1–10)	Justification
Damage Potential		
Reproducibility		
Exploitability		
Affected Users		
Discoverability		

Total Score: _____ / 50
Priority Level: High / Medium / Low

Section C: Short Answer Questions (1–2 pages)

1. Define the term data lifecycle and list all its stages.
2. Explain the difference between data sovereignty and data residency.
3. Why is data classification important in a cloud environment?
4. Give five real world example of each of the following classification levels:

    I. Public
    II. Internal
    III. Confidential
    IV. Highly sensitive

Section D: (1–2 pages)

1. Briefly explain why cloud compliance is more challenging than traditional IT compliance.
2. Choose two compliance standards (e.g., GDPR and HIPAA) and compare them using:
    - Who they apply to
    - The type of data they protect
    - A real-world consequence of non-compliance

3. A company stores customer data on multiple cloud platforms (AWS, Azure, GCP). List 3 steps it must take to remain compliant with international privacy laws.
4. Why is it important to understand data breach notification laws? What could happen if a company delays reporting a breach?

Section E: Lab Completion

Instructions: Attempt all questions. Provide command line steps, code snippets, and brief explanations where required with screeenshorts.

1: Encryption of Data at Rest Using OpenSSL

You are required to demonstrate how encryption can be applied to protect data at rest using OpenSSL.

a. Create a text file named secret.txt and write the following content inside: "Confidential data: Access is restricted to authorized users only."

b. Use OpenSSL to encrypt the file using the AES-256-CBC algorithm and save the output as secret.enc.

c. Decrypt the encrypted file and save the result as secret_decrypted.txt.

2: Encryption of Data in Transit Using HTTPS Server

You are required to demonstrate encryption of data in transit using HTTPS.

a. Generate a self signed SSL certificate and private key using OpenSSL, valid for 365 days. Save them as cert.pem and key.pem.

b. Create a Python script named https_server.py that uses the generated certificate and key to run a basic HTTPS server on port 4443.

c. Run the server and access https://127.0.0.1:4443 in your browser to verify secure communication.